

# Dehradun Waste Watch: Comprehensive Technical Documentation

## 1. Project Overview

**Dehradun Waste Watch** is an end-to-end AI system designed to modernize urban waste management. It consists of a sophisticated computer vision pipeline (MSSA-YOLOv8) and a robust web application built with FastAPI. The project addresses the “small object detection” challenge in waste management, identifying micro-waste like cigarette butts alongside macro-waste like plastic bottles.

---

## 2. File-by-File Deep Dive

### 2.1 app.py (The Orchestrator)

This is the main entry point of the web application. It handles the API routes, user authentication, and the complex inference pipeline.

**What it does:**

- **Inference Pipeline:** Instead of a simple model call, it uses a multi-stage process: 1. **SAHI (Slicing Aided Hyper Inference):** High-res images are sliced into patches to detect tiny objects. 2. **MSSA-YOLOv8:** The custom model processes these patches. 3. **COCO Gating:** A second YOLO model (trained on COCO) identifies people, cars, and animals. If a waste detection overlaps with a person (e.g., someone holding a bottle), it is suppressed to avoid false reports.
- **Authority Mapping:** It contains a hardcoded mapping of Dehradun localities to specific Nagar Nigam wards and representatives.
- **WebSocket Manager:** Tracks active connections to broadcast new reports to the admin dashboard in real-time.
- **Auth System:** Implements JWT (JSON Web Tokens) for secure citizen and authority login.

### 2.2 mssa\_layers.py (The Innovation)

This file defines the Multi-Scale Spatial Attention (MSSA) module, the core research contribution of the project.

**What it does:**

- **MSSA Module:** Captures features at three different scales using **Dilated Convolutions** (dilation rates 1, 2, and 4). This allows the model to see the “texture” of small waste and the “context” of large waste simultaneously.
- **Attention Mechanisms:**
  - **Channel Attention:** Focuses on “what” is in the image by recalibrating feature maps.
  - **Spatial Attention:** Focuses on “where” the objects are.
- **Dynamic Injection:** The `inject_mssa` function allows us to modify a standard YOLOv8 model at runtime without changing the original library code.

### 2.3 models.py (The Foundation)

Defines the database structure using SQLAlchemy.

**What it does:** - **User Table:** Stores user credentials, roles (Citizen, Authority, Admin), and organization details. - **WasteReport Table:** The heart of the data. It tracks the entire lifecycle: - **Reported** (Initial state) - **Taken Up** (Authority acknowledged) - **Being Solved** (Cleaning in progress) - **Solved** (Resolution photo uploaded) - **Relationships:** Uses foreign keys to link reports to the citizens who reported them and the authorities assigned to them.

### 2.4 train.py (The Brain)

Contains the logic for training the custom MSSA-YOLOv8 model.

**What it does:** - **Data Augmentation:** Uses a custom **Copy-Paste** strategy where micro-waste objects (like cigarette butts) are synthetically pasted onto random backgrounds to help the model learn rare classes. - **Hybrid Loss:** Combines **Focal Loss** (to handle class imbalance) and **CIoU Loss** (for precise bounding box regression). - **HPC Optimization:** Includes logic to detect H100 GPUs and optimize Tensor Core usage for faster training.

---

## 3. Key Algorithms & Concepts

### 3.1 SAHI (Slicing Aided Hyper Inference)

Standard YOLO models resize high-res images to 640x640, making small objects (like a 20px cigarette butt) disappear. SAHI slices the 4K/1080p image into 512x512 overlapping windows, runs inference on each, and merges the results using Non-Maximum Suppression (NMS).

### 3.2 MSSA (Multi-Scale Spatial Attention)

Standard convolutions have a fixed receptive field. By using parallel dilated convolutions, MSSA gathers information from a wider area without losing resolution. The attention maps then “highlight” the pixels most likely to contain waste.

### 3.3 COCO Gating

A significant problem in waste detection is “Context Errors” (e.g., a bottle in a shop window being flagged as litter). COCO Gating suppresses waste detections that have high IoU (Intersection over Union) with non-waste COCO categories like **person**, **car**, or **dining table**.

## 4. Anticipated Presentation Questions (Q&A)

### 4.1 Concept Questions

1. **Q: Why use Dilated Convolutions in the MSSA module?**
  - **A:** Dilated convolutions increase the receptive field without increasing the number of parameters or losing spatial resolution. This allows the model to understand the global context (where the waste is) while maintaining a high-resolution view of small details.
2. **Q: What is the benefit of Focal Loss over Standard Cross Entropy?**
  - **A:** Standard Cross Entropy is dominated by “easy” majority classes. Focal Loss adds a modulating factor  $(1 - p_t)^\gamma$  that reduces the loss contributed by easy examples and focuses the training on “hard” examples (like tiny, blurry waste objects).
3. **Q: How does the system handle “false positives” in a crowded street?**
  - **A:** Through COCO Gating. If a “plastic bottle” is detected, but it overlaps significantly with a “person” detected by the COCO model, the system assumes the person is carrying the bottle and suppresses the alert.

### 4.2 Code-Specific Questions

1. **Q: In `app.py`, how is a report assigned to an authority?**
    - **A:** Inside the `create_report` endpoint, we use the `locality` string from the form data. We then query the `User` table for a user with `role="authority"` whose `assigned_locality` matches the report’s locality.
  2. **Q: How do you “inject” the MSSA layers into the pre-trained YOLO model?**
    - **A:** In `mssa_layers.py`, the `inject_mssa` function iterates through the model’s sequential layers. When it finds a `C2f` block (at index 11 or higher in the neck), it replaces it with a `nn.Sequential` block containing the original `C2f` layer followed by our new MSSA module.
  3. **Q: Where is the “After Photo” stored when an authority marks a task as solved?**
    - **A:** It is handled in the `/api/authority/reports/{report_id}/update` endpoint. The image is saved to the `static/uploads` directory with a `solved_` prefix, and the path is stored in the `after_image_path` column of the `WasteReport` table.
-

## 5. Source Code Annex

app.py

```
import io
import os
import uuid
import logging
from datetime import datetime, timedelta
from typing import List, Optional
from contextlib import asynccontextmanager

from PIL import Image
from fastapi import FastAPI, File, UploadFile, HTTPException, Depends, Form, WebSocket, WebS
from fastapi.responses import JSONResponse
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy.orm import Session
from sqlalchemy import func
from ultralytics import YOLO
from passlib.context import CryptContext
import jwt
from mssa_layers import inject_mssa
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from sahi.models.ultralytics import UltralyticsDetectionModel
from sahi.predict import get_sliced_prediction
import numpy as np

from models import init_db, get_db, User, WasteReport

# Setup logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Config
SECRET_KEY = "supersecretkey_change_in_production"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60 * 24 * 7 # 1 week
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_DIR = os.path.join(BASE_DIR, "static", "uploads")
os.makedirs(UPLOAD_DIR, exist_ok=True)

# Dehradun localities mapped to responsible government authorities
DEHRADUN_AUTHORITIES = {
    "Rajpur Road": "Nagar Nigam Dehradun - Ward 1",
    "Clock Tower": "Nagar Nigam Dehradun - Ward 5",
```

```

"Paltan Bazaar": "Nagar Nigam Dehradun - Ward 5",
"ISBT Dehradun": "Nagar Nigam Dehradun - Ward 12",
"Clement Town": "Cantonment Board Clement Town",
"Sahastradhara Road": "Nagar Nigam Dehradun - Ward 2",
"Mussoorie Road": "Nagar Nigam Dehradun - Ward 3",
"Race Course": "Nagar Nigam Dehradun - Ward 6",
"Ballupur": "Nagar Nigam Dehradun - Ward 7",
"Dalanwala": "Nagar Nigam Dehradun - Ward 8",
"Karanpur": "Nagar Nigam Dehradun - Ward 9",
"Rispana": "Nagar Nigam Dehradun - Ward 10",
"Dharampur": "Nagar Nigam Dehradun - Ward 11",
"Nehru Colony": "Nagar Nigam Dehradun - Ward 13",
"Shimla Bypass": "Nagar Nigam Dehradun - Ward 14",
"Jakhan": "Nagar Nigam Dehradun - Ward 15",
"Chandrabani": "Nagar Nigam Dehradun - Ward 16",
"Prem Nagar": "Nagar Nigam Dehradun - Ward 17",
"Doiwala": "Nagar Palika Doiwala",
"Raipur": "Nagar Nigam Dehradun - Ward 18",
"Vasant Vihar": "Nagar Nigam Dehradun - Ward 19",
"Turner Road": "Nagar Nigam Dehradun - Ward 4",
"Subhash Road": "Nagar Nigam Dehradun - Ward 5",
"GMS Road": "Nagar Nigam Dehradun - Ward 20",
"IT Park": "Nagar Nigam Dehradun - Ward 21",
"Doon University Area": "Nagar Nigam Dehradun - Ward 22",
"Hathibarkala": "Nagar Nigam Dehradun - Ward 23",
"Kanwali": "Nagar Nigam Dehradun - Ward 24",
"Niranjanpur": "Nagar Nigam Dehradun - Ward 25",
"Other": "Nagar Nigam Dehradun - General"
}

```

```

DEHRADUN_REPRESENTATIVES = {
    "Rajpur Road": "Deepak Rawat (Councillor)",
    "Clock Tower": "Sanjeev Sharma (Councillor)",
    "Paltan Bazaar": "Sanjeev Sharma (Councillor)",
    "ISBT Dehradun": "Manoj Kumar (Councillor)",
    "Clement Town": "Sunil Rawat (Board Member)",
    "Sahastradhara Road": "Priyanka Negi (Councillor)",
    "Mussoorie Road": "Ravi Pathak (Councillor)",
    "Race Course": "Vicky Khurana (Councillor)",
    "Ballupur": "Anita Singh (Councillor)",
    "Dalanwala": "Meenakshi Maurya (Councillor)",
    "Karanpur": "Sanjay Nautiyal (Councillor)",
    "Rispana": "Kamleshwar Singh (Councillor)",
    "Dharampur": "Vinod Kumar (Councillor)",
    "Nehru Colony": "Rekha Mehra (Councillor)",
    "Shimla Bypass": "Mahesh Sharma (Councillor)",
}

```

```

        "Jakhan": "Pradeep Nautiyal (Councillor)",
        "Chandrabani": "Seema Bhardwaj (Councillor)",
        "Prem Nagar": "Surya Mani Tyagi (Councillor)",
        "Doiwala": "Sumit Bhardwaj (President)",
        "Raipur": "Rakesh Joshi (Councillor)",
        "Vasant Vihar": "Archana Purohit (Councillor)",
        "Turner Road": "Nirmala Bisht (Councillor)",
        "Subhash Road": "Sanjeev Sharma (Councillor)",
        "GMS Road": "Rahul Rana (Councillor)",
        "IT Park": "Swati Chandel (Councillor)",
        "Doon University Area": "Mohini Devi (Councillor)",
        "Hathibarkala": "Vipul Kumar (Councillor)",
        "Kanwali": "Ankit Saini (Councillor)",
        "Niranjanpur": "Devendra Singh (Councillor)",
        "Other": "Area Representative"
    }

DEHRADUN_LOCALITIES = list(DEHRADUN_AUTHORITIES.keys())
DEFAULT_LAT = 30.3165
DEFAULT_LNG = 78.0322

# Report status stages
REPORT_STAGES = ["Reported", "Taken Up", "Being Solved", "Solved"]

# Auth setup
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")

def verify_password(plain, hashed):
    return pwd_context.verify(plain, hashed)

def get_password_hash(password):
    return pwd_context.hash(password)

def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=15))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)):
    cred_exc = HTTPException(status_code=401, detail="Could not validate credentials",
                              headers={"WWW-Authenticate": "Bearer"})
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        email = payload.get("sub")

```

```

        if not email:
            raise cred_exc
    except jwt.PyJWTError:
        raise cred_exc
    user = db.query(User).filter(User.email == email).first()
    if not user:
        raise cred_exc
    return user

def require_admin(current_user: User = Depends(get_current_user)):
    if current_user.role != "admin":
        raise HTTPException(status_code=403, detail="Admin access required")
    return current_user

def require_authority(current_user: User = Depends(get_current_user)):
    if current_user.role not in ("authority", "admin"):
        raise HTTPException(status_code=403, detail="Authority access required")
    return current_user

@asynccontextmanager
async def lifespan(app: FastAPI):
    init_db()
    # Seed default admin account
    from models import SessionLocal
    db = SessionLocal()
    try:
        admin = db.query(User).filter(User.email == "admin@wastewatch.in").first()
        if not admin:
            admin = User(
                email="admin@wastewatch.in",
                password_hash=get_password_hash("admin123"),
                role="admin",
                name="System Admin"
            )
            db.add(admin)
            db.commit()
            logger.info("Seeded default admin account: admin@wastewatch.in / admin123")
    finally:
        db.close()
    yield

app = FastAPI(title="Dehradun Waste Watch API", lifespan=lifespan)

app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=True,
                  allow_methods=["*"], allow_headers=["*"])

```

```

# Load YOLO model
MODEL_PATH = os.path.join(BASE_DIR, "best.pt")
try:
    model = YOLO(MODEL_PATH)
    model = inject_mssa(model, device='cpu')
    logger.info(f"Successfully loaded and modified YOLO model with MSSA from {MODEL_PATH}")
except Exception as e:
    logger.error(f"Failed to load model from {MODEL_PATH}: {e}")
    model = YOLO("yolov8n.pt")
    model = inject_mssa(model, device='cpu')
    logger.warning("Fell back to standard yolov8n.pt model with MSSA")

# COCO gating model for reducing false positives
try:
    COCO_MODEL = YOLO('yolov8m.pt')
    NON_WASTE_COCO_IDS = {
        14, 15, 16, 17, 18, 19, 20, 21, 22, 23, # animals
        0, # person
        1, 2, 3, 4, 5, 6, 7, 8, # vehicles
        56, 57, 58, 59, 60, 61, 62, 63, 64, 65, # furniture
        46, 47, 48, 49, 50, 51, 52, 53, 54, 55, # food
    }
    logger.info("Successfully re-enabled COCO gating model (yolov8m.pt)")
except Exception as e:
    logger.warning(f"Failed to load COCO gating model: {e}")
    COCO_MODEL = None

# WebSocket manager
class ConnectionManager:
    def __init__(self):
        self.active_connections: List[WebSocket] = []
    async def connect(self, ws: WebSocket):
        await ws.accept()
        self.active_connections.append(ws)
    def disconnect(self, ws: WebSocket):
        self.active_connections.remove(ws)
    async def broadcast(self, msg: dict):
        for c in self.active_connections:
            try:
                await c.send_json(msg)
            except:
                pass

# EMAIL NOTIFICATION SYSTEM

```



```

def send_email_notification(to_email: str, subject: str, body: str):
    """
    Simulates sending an email.
    In a real-world scenario, you would use smtplib or fastapi-mail here.
    """
    logger.info(f" SENDING EMAIL TO: {to_email}")
    logger.info(f" SUBJECT: {subject}")
    logger.info(f" BODY:\n{body}\n{'-'*30}")

    # Real implementation example (commented out):
    # import smtplib
    # from email.mime.text import MIMEText
    # msg = MIMEText(body)
    # msg['Subject'] = subject
    # msg['From'] = "alerts@wastewatch.in"
    # msg['To'] = to_email
    # with smtplib.SMTP("smtp.gmail.com", 587) as server:
    #     server.starttls()
    #     server.login("your-email@gmail.com", "your-app-password")
    #     server.send_message(msg)

def notify_new_report(reporter_email: str, locality: str, waste_type: str, authorities: List):
    # Notify Reporter
    send_email_notification(
        reporter_email,
        "Waste Report Received - Dehradun Waste Watch",
        f"Hello,\n\nYour report for '{waste_type}' in '{locality}' has been received. "
        f"Our team is looking into it.\n\nThank you for helping keep Dehradun clean!"
    )
    # Authority notification removed for now

def notify_status_change(reporter_email: str, report_id: int, new_status: str):
    send_email_notification(
        reporter_email,
        f"Update on Your Waste Report #{report_id}",
        f"Hello,\n\nThe status of your waste report #{report_id} has been updated to: '{new_status}' "
        f"You can track the progress on our website."
    )

    async def broadcast(self, msg: dict):
        for c in self.active_connections:
            try:
                await c.send_json(msg)
            except:
                pass

```

```

manager = ConnectionManager()

@app.websocket("/api/ws/detections")
async def websocket_endpoint(ws: WebSocket):
    await manager.connect(ws)
    try:
        while True:
            await ws.receive_text()
    except WebSocketDisconnect:
        manager.disconnect(ws)

#         AUTH ROUTES

@app.post("/api/auth/register")
def register(
    email: str = Form(...),
    password: str = Form(...),
    role: str = Form("citizen"),
    name: str = Form(None),
    organization_name: str = Form(None),
    organization_type: str = Form(None),
    assigned_locality: str = Form(None),
    phone: str = Form(None),
    db: Session = Depends(get_db)
):
    if db.query(User).filter(User.email == email).first():
        raise HTTPException(status_code=400, detail="Email already registered")

    # Auto-detect admin: email starting with 'admin' gets admin role
    if email.lower().startswith("admin"):
        role = "admin"

    new_user = User(
        email=email,
        password_hash=get_password_hash(password),
        role=role,
        name=name or email.split("@")[0],
        organization_name=organization_name,
        organization_type=organization_type,
        assigned_locality=assigned_locality,
        phone=phone
    )
    db.add(new_user)
    db.commit()
    db.refresh(new_user)

```

```

        token = create_access_token(data={"sub": new_user.email},
                                     expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    return {
        "access_token": token, "token_type": "bearer",
        "user": {"id": new_user.id, "email": new_user.email, "role": new_user.role, "name":
                  "organization_name": new_user.organization_name, "assigned_locality": new_u
    }

@app.post("/api/auth/login")
def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == form_data.username).first()
    if not user or not verify_password(form_data.password, user.password_hash):
        raise HTTPException(status_code=400, detail="Incorrect email or password")

    token = create_access_token(data={"sub": user.email},
                                 expires_delta=timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    return {
        "access_token": token, "token_type": "bearer",
        "user": {
            "id": user.id, "email": user.email, "role": user.role, "name": user.name,
            "organization_name": user.organization_name, "assigned_locality": user.assigned
        }
    }

@app.get("/api/auth/me")
def get_me(current_user: User = Depends(get_current_user)):
    return {
        "id": current_user.id, "email": current_user.email, "role": current_user.role,
        "name": current_user.name, "organization_name": current_user.organization_name,
        "assigned_locality": current_user.assigned_locality
    }

#         ANALYZE (inference only)

@app.post("/api/analyze")
async def analyze_image(file: UploadFile = File(...)):
    """AI inference only - no DB save."""
    if not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="File uploaded is not an image.")
    try:
        contents = await file.read()
        image = Image.open(io.BytesIO(contents)).convert("RGB")
        # Use SAHI for better detection
        predictions, best_conf, best_class = get_sahi_predictions(image)

        # Re-enabled COCO gating for better suppression

```

```

predictions = apply_coco_gating(predictions, np.array(image))

if not predictions:
    return JsonResponse(content={"predictions": [], "message": "No waste detected at"})

# Recalculate best after gating
best_pred = max(predictions, key=lambda x: x["confidence"])
best_class = best_pred["class"]
best_conf = best_pred["confidence"]

# Micro-waste categories
MICRO_WASTE_CLASSES = ["cigarette_butt", "straw", "food_wrapper", "plastic_bottle_cap"]
severity = "micro-waste" if best_class in MICRO_WASTE_CLASSES else "macro-waste"

return JsonResponse(content={"predictions": predictions, "waste_type": best_class,
                             "all_detected": list(set(p["class"] for p in predictions)),
                             "confidence": best_conf, "severity": severity})

except Exception as e:
    logger.error(f"Analysis error: {e}")
    raise HTTPException(status_code=500, detail="Error analyzing image.")

# SAHI WRAPPER

def get_sahi_predictions(image_pil):
    """Run Slicing Aided Hyper Inference for better small object detection."""
    # Convert PIL to numpy (RGB)
    image_np = np.array(image_pil)

    # Use UltralyticsDetectionModel directly instead of the factory
    detection_model = UltralyticsDetectionModel(
        model=model, # Uses the injected MSSA-YOLO model
        confidence_threshold=0.25, # Increased from 0.15 to reduce false positives
        device='cpu' # Using CPU as default for compatibility
    )

    # Run sliced prediction
    result = get_sliced_prediction(
        image_np,
        detection_model,
        slice_height=512,
        slice_width=512,
        overlap_height_ratio=0.2,
        overlap_width_ratio=0.2,
        verbose=0
    )

```

```

predictions = []
best_conf = 0
best_class = "Unknown"

for object_prediction in result.object_prediction_list:
    bbox = object_prediction.bbox.to_xyxy()
    conf = object_prediction.score.value
    cls_name = object_prediction.category.name

    predictions.append({
        "class": cls_name,
        "confidence": float(conf),
        "bbox": [float(x) for x in bbox]
    })

    if conf > best_conf:
        best_conf = conf
        best_class = cls_name

return predictions, best_conf, best_class

def calculate_iou(boxA, boxB):
    """Compute IoU between two [x1,y1,x2,y2] boxes."""
    xA = max(boxA[0], boxB[0]); yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2]); yB = min(boxA[3], boxB[3])
    inter = max(0, xB - xA) * max(0, yB - yA)
    if inter == 0: return 0.0
    aA = (boxA[2]-boxA[0]) * (boxA[3]-boxA[1])
    aB = (boxB[2]-boxB[0]) * (boxB[3]-boxB[1])
    return inter / (float(aA + aB - inter) + 1e-7)

def apply_coco_gating(predictions, image_np, coco_iou_gate=0.5):
    """
    Suppresses waste detections that overlap with COCO non-waste objects.
    Ported from train.py logic.
    """
    if COCO_MODEL is None or not predictions:
        return predictions

    # Run COCO model on the image
    coco_results = COCO_MODEL.predict(source=image_np, conf=0.4, iou=0.45, imgsiz=640, verbose=0)

    coco_non_waste_boxes = []
    for box in coco_results.bboxes:
        if int(box.cls) in NON_WASTE_COCO_IDS:
            coco_non_waste_boxes.append(box.xyxy[0].tolist())

```

```

if not coco_non_waste_boxes:
    logger.info(f"COCO gating: No non-waste objects found. Keeping all {len(predictions)}")
    return predictions

filtered_preds = []
for pred in predictions:
    waste_box = pred["bbox"] # [x1, y1, x2, y2]
    suppressed = any(calculate_iou(waste_box, coco_box) > coco_iou_gate for coco_box in coco_boxes)
    if not suppressed:
        filtered_preds.append(pred)

logger.info(f"COCO gating: Filtered {len(predictions)} -> {len(filtered_preds)} detections")
return filtered_preds

# REPORT ROUTES

@app.post("/api/reports")
async def create_report(
    file: UploadFile = File(...),
    lat: float = Form(0.0),
    lng: float = Form(0.0),
    locality: str = Form("Other"),
    token: Optional[str] = Form(None),
    background_tasks: BackgroundTasks = BackgroundTasks(),
    db: Session = Depends(get_db)
):
    if not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="Not an image.")
    try:
        contents = await file.read()
        filename = f"{uuid.uuid4()}_{file.filename}"
        filepath = os.path.join(UPLOAD_DIR, filename)
        with open(filepath, "wb") as f:
            f.write(contents)

        image = Image.open(io.BytesIO(contents)).convert("RGB")
        # Use SAHI for better detection
        predictions, best_conf, best_class = get_sahi_predictions(image)

        # Re-enabled COCO gating for better suppression
        predictions = apply_coco_gating(predictions, np.array(image))

    except Exception as e:
        logger.error(f"Error in create_report: {e}")

    if not predictions:
        return JSONResponse(content={"predictions": [], "message": "No waste detected at this location"})

```

```

# Recalculate best after gating
best_pred = max(predictions, key=lambda x: x["confidence"])
best_class = best_pred["class"]
best_conf = best_pred["confidence"]

MICRO_WASTE_CLASSES = ["cigarette_butt", "straw", "food_wrapper", "plastic_bottle_cap"]
severity = "micro-waste" if best_class in MICRO_WASTE_CLASSES else "macro-waste"

# Resolve reporter
reporter_id = None
if token:
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user = db.query(User).filter(User.email == payload.get("sub")).first()
        if user: reporter_id = user.id
    except:
        pass

# Auto-assign authority based on locality
authority_id = None
authority = db.query(User).filter(User.role == "authority", User.assigned_locality == locality).first()
if authority:
    authority_id = authority.id

report = WasteReport(
    lat=lat, lng=lng, locality=locality,
    image_path=f"/uploads/{filename}",
    waste_type=best_class, severity=severity, confidence=best_conf,
    all_detected_objects=", ".join(list(set(p["class"] for p in predictions))),
    reporter_id=reporter_id, authority_id=authority_id,
    status="Reported"
)
db.add(report)
db.commit()
db.refresh(report)

# Email Notifications via Background Task
if report.reporter:
    background_tasks.add_task(
        notify_new_report,
        report.reporter.email,
        locality,
        best_class,
        [] # Authority notification removed
    )

```

```

        await manager.broadcast({
            "type": "new_report",
            "report": {
                "id": report.id, "lat": report.lat, "lng": report.lng,
                "waste_type": report.waste_type, "severity": report.severity,
                "status": report.status, "locality": report.locality
            }
        })

    return {
        "id": report.id, "waste_type": best_class, "confidence": best_conf,
        "all_detected_objects": report.all_detected_objects,
        "responsible_authority": DEHRADUN_AUTHORITIES.get(locality, "Unassigned"),
        "representative_name": DEHRADUN_REPRESENTATIVES.get(locality, "Area Representative"),
        "locality": locality, "status": "Reported"
    }
except Exception as e:
    logger.error(f"Report creation error: {e}")
    raise HTTPException(status_code=500, detail="Internal error during report creation.")

@app.get("/api/reports")
def get_reports(db: Session = Depends(get_db)):
    reports = db.query(WasteReport).order_by(WasteReport.reported_at.desc()).all()
    return [_serialize_report(r) for r in reports]

@app.get("/api/reports/{report_id}")
def get_report(report_id: int, db: Session = Depends(get_db)):
    r = db.query(WasteReport).filter(WasteReport.id == report_id).first()
    if not r: raise HTTPException(404, "Report not found")
    return _serialize_report(r)

def _serialize_report(r):
    return {
        "id": r.id, "lat": r.lat, "lng": r.lng, "locality": r.locality,
        "image_path": r.image_path, "waste_type": r.waste_type,
        "all_detected_objects": r.all_detected_objects or r.waste_type,
        "severity": r.severity, "confidence": r.confidence,
        "status": r.status,
        "reported_at": r.reported_at.isoformat() if r.reported_at else None,
        "taken_up_at": r.taken_up_at.isoformat() if r.taken_up_at else None,
        "being_solved_at": r.being_solved_at.isoformat() if r.being_solved_at else None,
        "solved_at": r.solved_at.isoformat() if r.solved_at else None,
        "admin_notes": r.admin_notes,
        "resolution_notes": r.resolution_notes,
        "after_image_path": r.after_image_path,
        "reporter_email": r.reporter.email if r.reporter else "Anonymous",
    }

```



```

        "authority_name": r.assigned_authority.organization_name if r.assigned_authority else None,
        "representative_name": DEHRADUN_REPRESENTATIVES.get(r.locality, "Area Representative"),
        "authority_id": r.authority_id,
        "user_verified": r.user_verified,
        "verification_comment": r.verification_comment
    }

# ADMIN ROUTES

@app.post("/api/admin/reports/{report_id}/status")
async def update_report_status(
    report_id: int,
    status: str = Form(...),
    notes: str = Form(None),
    file: Optional[UploadFile] = File(None),
    current_user: User = Depends(require_admin),
    background_tasks: BackgroundTasks = BackgroundTasks(),
    db: Session = Depends(get_db)
):
    if status not in REPORT_STAGES:
        raise HTTPException(400, f"Invalid status. Must be one of: {REPORT_STAGES}")

    report = db.query(WasteReport).filter(WasteReport.id == report_id).first()
    if not report: raise HTTPException(404, "Report not found")

    report.status = status
    if notes: report.admin_notes = notes

    now = datetime.utcnow()
    if status == "Taken Up": report.taken_up_at = now
    elif status == "Being Solved": report.being_solved_at = now
    elif status == "Solved":
        report.solved_at = now
        if file and file.content_type and file.content_type.startswith("image/"):
            contents = await file.read()
            fname = f"resolved_{uuid.uuid4()}_{file.filename}"
            fpath = os.path.join(UPLOAD_DIR, fname)
            with open(fpath, "wb") as f:
                f.write(contents)
            report.after_image_path = f"/uploads/{fname}"

    db.commit()

# Notify Reporter of Status Change
if report.reporter:
    background_tasks.add_task(notify_status_change, report.reporter.email, report.id, status)

```

```

        return {"message": f"Report #{report_id} status updated to '{status}'"}

@app.post("/api/admin/reports/{report_id}/assign")
def assign_authority(
    report_id: int,
    authority_id: int = Form(...),
    current_user: User = Depends(require_admin),
    db: Session = Depends(get_db)
):
    report = db.query(WasteReport).filter(WasteReport.id == report_id).first()
    if not report: raise HTTPException(404, "Report not found")

    authority = db.query(User).filter(User.id == authority_id, User.role == "authority").first()
    if not authority: raise HTTPException(404, "Authority user not found")

    report.authority_id = authority_id
    db.commit()
    return {"message": f"Assigned to {authority.organization_name}"}

@app.get("/api/admin/authorities")
def list_authorities(current_user: User = Depends(require_admin), db: Session = Depends(get_db)):
    auths = db.query(User).filter(User.role == "authority").all()
    return [{
        "id": a.id, "name": a.name, "email": a.email,
        "organization_name": a.organization_name,
        "assigned_locality": a.assigned_locality
    } for a in auths]

@app.post("/api/admin/authorities")
def create_authority(
    email: str = Form(...),
    password: str = Form(...),
    name: str = Form(...),
    organization_name: str = Form(...),
    assigned_locality: str = Form(...),
    phone: str = Form(None),
    current_user: User = Depends(require_admin),
    db: Session = Depends(get_db)
):
    if db.query(User).filter(User.email == email).first():
        raise HTTPException(400, "Email already registered")

    auth_user = User(
        email=email, password_hash=get_password_hash(password),
        role="authority", name=name,

```

```

        organization_name=organization_name,
        assigned_locality=assigned_locality, phone=phone
    )
    db.add(auth_user)
    db.commit()
    db.refresh(auth_user)
    return {"id": auth_user.id, "message": f"Authority '{name}' created for {assigned_locality}"}

#         AUTHORITY ROUTES

@app.get("/api/authority/reports")
def get_authority_reports(current_user: User = Depends(require_authority), db: Session = Depends(get_db)):
    """Get reports assigned to this authority."""
    if current_user.role == "admin":
        reports = db.query(WasteReport).order_by(WasteReport.reported_at.desc()).all()
    else:
        reports = db.query(WasteReport).filter(
            WasteReport.authority_id == current_user.id
        ).order_by(WasteReport.reported_at.desc()).all()
    return [_serialize_report(r) for r in reports]

@app.post("/api/authority/reports/{report_id}/update")
async def authority_update_report(
    report_id: int,
    status: str = Form(...),
    notes: str = Form(None),
    file: Optional[UploadFile] = File(None),
    current_user: User = Depends(require_authority),
    background_tasks: BackgroundTasks = BackgroundTasks(),
    db: Session = Depends(get_db)
):
    report = db.query(WasteReport).filter(WasteReport.id == report_id).first()
    if not report: raise HTTPException(404, "Report not found")

    # Verify authority owns this report (or is admin)
    if current_user.role != "admin" and report.authority_id != current_user.id:
        raise HTTPException(403, "Not assigned to this report")

    if status not in REPORT_STAGES:
        raise HTTPException(400, f"Invalid status. Must be one of: {REPORT_STAGES}")

    report.status = status
    if notes: report.resolution_notes = notes

    now = datetime.utcnow()
    if status == "Taken Up": report.taken_up_at = now

```

```

elif status == "Being Solved": report.being_solved_at = now
elif status == "Solved":
    report.solved_at = now
    # Handle after photo
    if file and file.content_type.startswith("image/"):
        contents = await file.read()
        fname = f"solved_{uuid.uuid4()}_{file.filename}"
        fpath = os.path.join(UPLOAD_DIR, fname)
        with open(fpath, "wb") as f:
            f.write(contents)
        report.after_image_path = f"/uploads/{fname}"

db.commit()

# Notify Reporter of Status Change
if report.reporter:
    background_tasks.add_task(notify_status_change, report.reporter.email, report.id, status)

return {"message": f"Report #{report_id} updated to '{status}'"}

# VERIFICATION ROUTES

@app.post("/api/reports/{report_id}/verify")
def verify_report(
    report_id: int,
    verified: bool = Form(...),
    comment: str = Form(None),
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    report = db.query(WasteReport).filter(WasteReport.id == report_id).first()
    if not report: raise HTTPException(404, "Report not found")
    if report.status != "Solved":
        raise HTTPException(400, "Only solved reports can be verified")

    report.user_verified = verified
    report.verified_by_id = current_user.id
    report.verification_comment = comment
    db.commit()
    return {"message": f"Report #{report_id} verification: {'Confirmed' if verified else 'Denied'}"}

# AREA GALLERY

@app.get("/api/reports/area/{locality}")
def get_area_reports(locality: str, db: Session = Depends(get_db)):
    """Get all reports for a specific locality - for area gallery dropdown."""

```

```

        reports = db.query(WasteReport).filter(
            WasteReport.locality == locality
        ).order_by(WasteReport.reported_at.desc()).all()
        return [_serialize_report(r) for r in reports]

#         DATA ENDPOINTS

@app.get("/api/localities")
def get_localities():
    return {
        "localities": DEHRADUN_LOCALITIES,
        "authorities": DEHRADUN_AUTHORITIES,
        "representatives": DEHRADUN_REPRESENTATIVES,
        "default_lat": DEFAULT_LAT,
        "default_lng": DEFAULT_LNG
    }

@app.get("/api/stats")
def get_stats(db: Session = Depends(get_db)):
    total = db.query(WasteReport).count()
    reported = db.query(WasteReport).filter(WasteReport.status == "Reported").count()
    taken_up = db.query(WasteReport).filter(WasteReport.status == "Taken Up").count()
    being_solved = db.query(WasteReport).filter(WasteReport.status == "Being Solved").count()
    solved = db.query(WasteReport).filter(WasteReport.status == "Solved").count()

    area_stats = db.query(WasteReport.locality, func.count(WasteReport.id)
        ).group_by(WasteReport.locality).all()
    type_stats = db.query(WasteReport.waste_type, func.count(WasteReport.id)
        ).group_by(WasteReport.waste_type).all()

    return {
        "total": total, "reported": reported, "taken_up": taken_up,
        "being_solved": being_solved, "solved": solved,
        "by_area": {a: c for a, c in area_stats if a},
        "by_type": {t: c for t, c in type_stats if t}
    }

# Mount static files last
app.mount("/", StaticFiles(directory=os.path.join(BASE_DIR, "static"), html=True), name="static")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)

```

models.py

```
from sqlalchemy import create_engine, Column, Integer, String, Float, ForeignKey, DateTime,
from sqlalchemy.orm import declarative_base, relationship, sessionmaker
from datetime import datetime
import os as _os
```

```
Base = declarative_base()
```

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True)
    password_hash = Column(String)
    role = Column(String) # "citizen", "admin", "authority"
    name = Column(String, nullable=True)

    # Authority specific fields
    organization_name = Column(String, nullable=True)
    organization_type = Column(String, nullable=True)
    assigned_locality = Column(String, nullable=True)
    phone = Column(String, nullable=True)

    reports = relationship("WasteReport", back_populates="reporter", foreign_keys="WasteReport")
    assigned_reports = relationship("WasteReport", back_populates="assigned_authority", foreign_keys="WasteReport")
```

```
class WasteReport(Base):
    __tablename__ = "waste_reports"

    id = Column(Integer, primary_key=True, index=True)
    lat = Column(Float, index=True)
    lng = Column(Float, index=True)
    image_path = Column(String, nullable=True)
    waste_type = Column(String)
    severity = Column(String)
    confidence = Column(Float)
    locality = Column(String, nullable=True)
    all_detected_objects = Column(Text, nullable=True)

    # Report lifecycle
    status = Column(String, default="Reported") # Reported → Taken Up → Being Solved → Solved

    # Timestamps per stage
    reported_at = Column(DateTime, default=datetime.utcnow)
```

```

taken_up_at = Column(DateTime, nullable=True)
being_solved_at = Column(DateTime, nullable=True)
solved_at = Column(DateTime, nullable=True)

# Notes
admin_notes = Column(Text, nullable=True)
resolution_notes = Column(Text, nullable=True)
after_image_path = Column(String, nullable=True)

# User verification of resolution
user_verified = Column(Boolean, default=False)
verified_by_id = Column(Integer, ForeignKey("users.id"), nullable=True)
verification_comment = Column(Text, nullable=True)

# Relationships
reporter_id = Column(Integer, ForeignKey("users.id"), nullable=True)
reporter = relationship("User", back_populates="reports", foreign_keys=[reporter_id])

authority_id = Column(Integer, ForeignKey("users.id"), nullable=True)
assigned_authority = relationship("User", back_populates="assigned_reports", foreign_key

# Database setup
_db_path = _os.path.join(_os.path.dirname(_os.path.abspath(__file__)), "waste_app.db")
SQLALCHEMY_DATABASE_URL = f"sqlite:/// {_db_path}"
engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

def init_db():
    Base.metadata.create_all(bind=engine)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

mssa_layers.py

import torch
import torch.nn as nn
import torch.nn.functional as F

class ChannelAttention(nn.Module):
    """Squeeze-and-Excitation channel attention (avg + max pool fusion)."""

```

```

def __init__(self, in_channels, reduction=16):
    super().__init__()
    mid = max(1, in_channels // reduction)
    self.avg_pool = nn.AdaptiveAvgPool2d(1)
    self.max_pool = nn.AdaptiveMaxPool2d(1)
    self.fc = nn.Sequential(
        nn.Flatten(),
        nn.Linear(in_channels, mid, bias=False),
        nn.ReLU(inplace=True),
        nn.Linear(mid, in_channels, bias=False),
    )
    self.sigmoid = nn.Sigmoid()

def forward(self, x):
    avg_out = self.fc(self.avg_pool(x))
    max_out = self.fc(self.max_pool(x))
    return self.sigmoid(avg_out + max_out).unsqueeze(-1).unsqueeze(-1) * x

class SpatialAttention(nn.Module):
    """Spatial attention using avg + max pooled channel maps."""
    def __init__(self, kernel_size=7):
        super().__init__()
        pad = kernel_size // 2
        self.conv = nn.Conv2d(2, 1, kernel_size, padding=pad, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        attn = self.conv(torch.cat([avg_out, max_out], dim=1))
        return self.sigmoid(attn) * x

class MSSA(nn.Module):
    """
    Multi-Scale Spatial Attention Module.
    """
    def __init__(self, in_channels, reduction=16):
        super().__init__()
        mid = in_channels // 4

        self.scale1 = nn.Sequential(
            nn.Conv2d(in_channels, mid, 1, bias=False),
            nn.BatchNorm2d(mid), nn.SiLU()
        )

```



```

self.scale2 = nn.Sequential(
    nn.Conv2d(in_channels, mid, 3, padding=2, dilation=2, bias=False),
    nn.BatchNorm2d(mid), nn.SiLU()
)
self.scale3 = nn.Sequential(
    nn.Conv2d(in_channels, mid, 3, padding=4, dilation=4, bias=False),
    nn.BatchNorm2d(mid), nn.SiLU()
)

self.fuse = nn.Sequential(
    nn.Conv2d(mid * 3, in_channels, 1, bias=False),
    nn.BatchNorm2d(in_channels),
)

self.channel_attn = ChannelAttention(in_channels, reduction)
self.spatial_attn = SpatialAttention(kernel_size=7)
self.act = nn.SiLU()

def forward(self, x):
    s1 = self.scale1(x)
    s2 = self.scale2(x)
    s3 = self.scale3(x)
    fused = self.fuse(torch.cat([s1, s2, s3], dim=1))
    out = self.channel_attn(fused)
    out = self.spatial_attn(out)
    return self.act(out + x)

def inject_mssa(yolo_model, device='cpu'):
    """
    Injects MSSA after each C2f block in the YOLOv8 neck (layers 11+).
    """
    from ultralytics.nn.modules import C2f

    seq = yolo_model.model.model
    injected = []

    for idx, layer in enumerate(seq):
        if isinstance(layer, C2f) and idx >= 11:
            try:
                in_ch = layer.cv2.conv.out_channels
            except AttributeError:
                continue
            mssa = MSSA(in_channels=in_ch).to(device)
            new_seq = nn.Sequential(layer, mssa)
            # Crucial: Copy Ultralytics metadata to the new Sequential block
            new_seq.i = idx

```

```

        new_seq.f = layer.f
        new_seq.type = layer.type

        seq[idx] = new_seq
        injected.append(idx)

    if injected:
        print(f"MSSA injected at neck layers: {injected}")
    else:
        print("No layers injected with MSSA. Check model architecture.")

    return yolo_model

```

## train.py

```

import os
os.environ['WANDB_DISABLED'] = 'true'
os.environ['WANDB_MODE'] = 'disabled'

# =====
# model_B.py - MSSA-YOLOv8 Waste Detection (Colab)
# =====

# 0. Dependencies
# Run once in terminal before executing this file:
# pip install "numpy==1.26.4" "ultralalytics==8.3.0" "scikit-image==0.21.0" \
#           "albumentations==1.4.18" torchinfo supervision einops

# =====
# 1.2 Imports and global config
# =====

import os, shutil, json, random, math, time, warnings, zipfile
from pathlib import Path
from collections import defaultdict

import numpy as np
import cv2
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import matplotlib.patches as mpatches
from PIL import Image

import torch
import torch.nn as nn
import torch.nn.functional as F

```

```

import torchvision.transforms as T

import albumentations as A
from albumentations.pytorch import ToTensorV2

from ultralytics import YOLO
import yaml

warnings.filterwarnings('ignore')

# Paths & Project Config
PROJECT_ROOT = Path(__file__).parent.resolve()
BASE_DIR = PROJECT_ROOT
DATA_DIR = BASE_DIR / 'data'
MODEL_DIR = BASE_DIR / 'models'
RUNS_DIR = BASE_DIR / 'runs'
OUTPUTS_DIR = BASE_DIR / 'outputs'

# Ensure project directories exist
for d in [DATA_DIR, MODEL_DIR, RUNS_DIR, OUTPUTS_DIR]:
    d.mkdir(parents=True, exist_ok=True)

# Use existing unzipped folder structure
UNIFIED_BASE = PROJECT_ROOT / 'unified_waste_dataset'
UNIFIED = UNIFIED_BASE / 'unified'
UNIFIED_TRAIN = UNIFIED / 'train'
UNIFIED_VAL = UNIFIED / 'val'
UNIFIED_TEST = UNIFIED / 'test'
YAML_PATH = UNIFIED_BASE / 'unified_waste.yaml'

UNIFIED_ZIP = PROJECT_ROOT / 'unified_waste_dataset.zip'

# H100 HPC Optimization
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

if DEVICE == 'cuda':
    # Optimize for Tensor Cores on H100
    torch.set_float32_matmul_precision('high')
    # Enable benchmark for faster execution
    torch.backends.cudnn.benchmark = True
    print(f'H100/GPU Optimized: matmul_precision=high, cudnn_benchmark=True')

print(f'Device : {DEVICE}')
print(f'PyTorch : {torch.__version__}')
print(f'Project Root : {PROJECT_ROOT}')
print(f'Outputs Dir : {OUTPUTS_DIR}')

```

```

print(f'YAML Path      : {YAML_PATH}')
print()

# =====
# 2.1 Load unified dataset
# =====

def dataset_already_extracted():
    img_dir = UNIFIED_TRAIN / 'images'
    return (
        img_dir.exists() and
        any(img_dir.glob('*.jpg')) and
        YAML_PATH.exists()
    )

def extract_unified_zip(zip_path):
    """Extract unified_waste_dataset.zip into DATA_DIR."""
    zip_path = Path(zip_path)
    if not zip_path.exists():
        return False
    size_mb = os.path.getsize(zip_path) / 1e6
    print(f'    Extracting {zip_path.name} ({size_mb:.0f} MB)...')
    with zipfile.ZipFile(zip_path) as z:
        z.extractall(DATA_DIR)
    print(f'    Extracted to {DATA_DIR}')
    return True

if dataset_already_extracted():
    print('Unified dataset already present - skipping extraction')
    print(f'    Location: {UNIFIED}')

elif UNIFIED_ZIP.exists():
    print(f'Found ZIP at {UNIFIED_ZIP}')
    extract_unified_zip(UNIFIED_ZIP)

else:
    print(f'Dataset not found at {UNIFIED}')
    print(f'ZIP not found at {UNIFIED_ZIP}')
    print('Please ensure unified_waste_dataset.zip is in the project directory.')

# =====
# 2.2 Verify dataset and configure paths
# =====

if not YAML_PATH.exists():
    raise FileNotFoundError(

```

```

        f'YAML not found at {YAML_PATH}\n'
        f'Make sure the ZIP extracted correctly.'
    )

    with open(YAML_PATH) as f:
        cfg = yaml.safe_load(f)

    cfg['path'] = str(UNIFIED)

    with open(YAML_PATH, 'w') as f:
        yaml.dump(cfg, f, default_flow_style=False)

    WASTE_CLASSES = cfg['names']
    NUM_CLASSES = cfg['nc']

    n_train = len(list((UNIFIED_TRAIN / 'images').glob('*.jpg')))
    n_val = len(list((UNIFIED_VAL / 'images').glob('*.jpg')))
    n_test = len(list((UNIFIED_TEST / 'images').glob('*.jpg')))

    print('Dataset configured successfully')
    print()
    print(f' Train images : {n_train}')
    print(f' Val images : {n_val}')
    print(f' Test images : {n_test}')
    print(f' Total images : {n_train + n_val + n_test}')
    print(f' Classes ({NUM_CLASSES}) : {WASTE_CLASSES}')
    print(f' YAML path : {YAML_PATH}')

# =====
# 2.3 Dataset statistics
# =====
def dataset_stats(unified_dir):
    counts = {}
    cls_counts = defaultdict(int)
    src_counts = defaultdict(int)

    for split in ['train', 'val', 'test']:
        lbl_dir = Path(unified_dir) / split / 'labels'
        img_dir = Path(unified_dir) / split / 'images'
        if not lbl_dir.exists():
            continue
        imgs = list(img_dir.glob('*.jpg'))
        instances = 0
        for lbl in lbl_dir.glob('*.txt'):
            with open(lbl) as f:

```

```

        for line in f:
            parts = line.strip().split()
            if parts:
                instances += 1
                if split == 'train':
                    cls_counts[int(parts[0])] += 1
            counts[split] = {'images': len(imgs), 'instances': instances}

    for img in (Path(unified_dir) / 'train' / 'images').glob('*.jpg'):
        name = img.name.lower()
        if name.startswith('taco_'):
            src_counts['TACO'] += 1
        elif name.startswith('trashnet_'):
            src_counts['TrashNet'] += 1
        elif name.startswith('cigbutt_'):
            src_counts['CigButt'] += 1
        else:
            src_counts['Other'] += 1

    return counts, cls_counts, src_counts

counts, cls_counts, src_counts = dataset_stats(UNIFIED)

print(' DATASET SUMMARY')
for split, info in counts.items():
    print(f' {split.upper():<8} {info["images"]:>5} images | '
          f'{info["instances"]:>6} instances')
total_imgs = sum(v['images'] for v in counts.values())
total_inst = sum(v['instances'] for v in counts.values())
print(f' {"TOTAL":<8} {total_imgs:>5} images | {total_inst:>6} instances')
print()
print(' SOURCE BREAKDOWN (training set):')
for src, cnt in sorted(src_counts.items(), key=lambda x: -x[1]):
    pct = cnt / counts['train']['images'] * 100 if counts['train']['images'] > 0 else 0
    bar = ' ' * int(pct / 3)
    print(f' {src:<15} {cnt:>5} ({pct:.1f}%) {bar}')
print('=' * 55)

fig, axes = plt.subplots(1, 3, figsize=(20, 5))

counts_list = [cls_counts.get(i, 0) for i in range(NUM_CLASSES)]
colors = ['#e74c3c' if i >= 6 else '#3498db' for i in range(NUM_CLASSES)]
bars = axes[0].bar(range(NUM_CLASSES), counts_list,
                  color=colors, edgecolor='white')
axes[0].set_xticks(range(NUM_CLASSES))
axes[0].set_xticklabels(WASTE_CLASSES, rotation=45, ha='right', fontsize=8)

```

```

axes[0].set_title('Training Set - Instances per Class', fontweight='bold')
axes[0].set_ylabel('Label instances')
axes[0].grid(axis='y', alpha=0.3)
for bar, cnt in zip(bars, counts_list):
    if cnt > 0:
        axes[0].text(bar.get_x() + bar.get_width()/2, cnt + 1,
                      str(cnt), ha='center', va='bottom', fontsize=7)
axes[0].legend(handles=[
    mpatches.Patch(color='#3498db', label='Macro waste (cls 0-5)'),
    mpatches.Patch(color='#e74c3c', label='Micro waste (cls 6-9)')
])

split_imgs = [counts[s]['images'] for s in ['train', 'val', 'test']]
axes[1].pie(
    split_imgs,
    labels=[f'{s}\n{counts[s]["images"]} images\n{counts[s]["instances"]} instances'
            for s in ['train', 'val', 'test']],
    colors=['#3498db', '#f39c12', '#e74c3c'],
    autopct='%1.1f%%', startangle=90,
    textprops={'fontsize': 9}
)
axes[1].set_title('Dataset Split', fontweight='bold')

if src_counts:
    src_labels = list(src_counts.keys())
    src_values = list(src_counts.values())
    axes[2].pie(
        src_values,
        labels=[f'{s}\n{v} images' for s, v in zip(src_labels, src_values)],
        colors=['#3498db', '#2ecc71', '#e74c3c', '#9b59b6'][:len(src_labels)],
        autopct='%1.1f%%', startangle=90,
        textprops={'fontsize': 9}
    )
    axes[2].set_title('Training Images by Source', fontweight='bold')

plt.suptitle('Unified Waste Dataset - Overview', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.savefig(str(OUTPUTS_DIR / 'dataset_overview.png'), dpi=150, bbox_inches='tight')
plt.show()
print(f'Chart saved → {OUTPUTS_DIR}/dataset_overview.png')

# =====
# 2.4 Sample images with bounding boxes
# =====
CLASS_COLORS_BGR = {

```

```

0: (46, 204, 113), # plastic_bottle green
1: (52, 152, 219), # plastic_bag blue
2: (230, 126, 34), # cardboard orange
3: (155, 89, 182), # glass_bottle purple
4: (149, 165, 166), # metal_can grey
5: (241, 196, 15), # paper yellow
6: (231, 76, 60), # cigarette_butt red
7: ( 26, 188, 156), # food_wrapper teal
8: (200, 200, 200), # straw light grey
9: (127, 140, 141), # general_waste dark grey
}

class_to_imgs = defaultdict(list)
for lbl_path in sorted((UNIFIED_TRAIN / 'labels').glob('*.txt')):
    img_path = UNIFIED_TRAIN / 'images' / lbl_path.with_suffix('.jpg').name
    if not img_path.exists():
        continue
    seen = set()
    with open(lbl_path) as f:
        for line in f:
            parts = line.strip().split()
            if parts:
                seen.add(int(parts[0]))
    for cid in seen:
        class_to_imgs[cid].append(img_path)

selected = []
random.seed(42)
for cid in range(NUM_CLASSES):
    imgs = class_to_imgs.get(cid, [])
    random.shuffle(imgs)
    for img_path in imgs[:3]:
        selected.append((img_path, cid))

seen_paths = set()
deduped = []
for img_path, cid in selected:
    if str(img_path) not in seen_paths:
        deduped.append((img_path, cid))
        seen_paths.add(str(img_path))

COLS = 4
n_imgs = min(len(deduped), 24)
ROWS = (n_imgs + COLS - 1) // COLS

fig, axes = plt.subplots(ROWS, COLS, figsize=(COLS * 4, ROWS * 3.5))

```



```

axes = axes.flatten() if n_imgs > COLS else [axes] if ROWS == 1 else axes.flatten()

for ax_idx, (img_path, highlight_cls) in enumerate(deduped[:n_imgs]):
    ax = axes[ax_idx]
    img = cv2.imread(str(img_path))
    if img is None:
        ax.axis('off')
        continue
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    H, W = img.shape[:2]

    lbl_path = UNIFIED_TRAIN / 'labels' / img_path.with_suffix('.txt').name
    classes_in_img = []

    if lbl_path.exists():
        with open(lbl_path) as f:
            for line in f:
                parts = line.strip().split()
                if not parts:
                    continue
                cid = int(parts[0])
                cx, cy, bw, bh = map(float, parts[1:])
                x1 = max(0, int((cx - bw/2) * W))
                y1 = max(0, int((cy - bh/2) * H))
                x2 = min(W, int((cx + bw/2) * W))
                y2 = min(H, int((cy + bh/2) * H))
                color = CLASS_COLORS_BGR.get(cid, (255, 255, 255))
                thickness = 3 if cid == highlight_cls else 1
                cv2.rectangle(img, (x1, y1), (x2, y2), color, thickness)
                label = WASTE_CLASSES[cid] if cid < NUM_CLASSES else str(cid)
                (tw, th) = cv2.getTextSize(
                    label, cv2.FONT_HERSHEY_SIMPLEX, 0.35, 1)
                cv2.rectangle(img,
                    (x1, max(y1 - th - 4, 0)),
                    (x1 + tw + 4, y1), color, -1)
                cv2.putText(img, label,
                    (x1 + 2, max(y1 - 2, th)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.35, (0, 0, 0), 1)
                classes_in_img.append(label)

    ax.imshow(img)
    ax.axis('off')

    name = img_path.name.lower()
    src = ('TACO' if name.startswith('taco_') else
           'TrashNet' if name.startswith('trashnet_') else

```

```

        'CigButt' if name.startswith('cigbutt_') else '?')

    unique_cls = list(dict.fromkeys(classes_in_img))
    title = f'[{src}] {"", ".join(unique_cls[:2])}'
    if len(unique_cls) > 2:
        title += f' +{len(unique_cls)-2}'
    ax.set_title(title, fontsize=7, pad=3)

for ax in axes[n_imgs:]:
    ax.axis('off')

plt.suptitle(
    f'Sample Training Images with Ground Truth Bounding Boxes '
    f'({n_imgs} shown)',
    fontsize=12, fontweight='bold'
)
plt.tight_layout()
plt.savefig(str(OUTPUTS_DIR / 'sample_images.png'), dpi=150, bbox_inches='tight')
plt.show()
print(f'Sample grid saved → {OUTPUTS_DIR}/sample_images.png')

# =====
# 3.1 Augmentation pipeline (Albumentations)
# =====
train_transform = A.Compose([
    A.HorizontalFlip(p=0.5),
    A.Rotate(limit=15, p=0.5),
    A.RandomBrightnessContrast(
        brightness_limit=0.2, contrast_limit=0.2, p=0.6),
    A.HueSaturationValue(
        hue_shift_limit=10, sat_shift_limit=20, val_shift_limit=10, p=0.3),
    A.GaussianBlur(blur_limit=(3, 5), p=0.2),
    A.RandomShadow(p=0.2),
    A.CLAHE(p=0.2),
    A.Resize(640, 640),
], bbox_params=A.BboxParams(
    format='yolo',
    label_fields=['class_labels'],
    min_visibility=0.3
))

val_transform = A.Compose([
    A.Resize(640, 640),
], bbox_params=A.BboxParams(
    format='yolo',

```

```

        label_fields=['class_labels']
    ))

    print('Augmentation pipelines defined')
    print()
    print('Train augmentations:')
    for t in train_transform.transforms:
        print(f'    • {t.__class__.__name__}')

# =====
# 3.2 Copy-paste micro-waste augmentation
# =====
def copy_paste_micro_waste(img, labels, micro_crops,
                           paste_prob=0.5, n_paste=3):
    """
    Pastes micro-waste crops onto an image.
    Increases minority class presence synthetically.
    """
    if random.random() > paste_prob or len(micro_crops) == 0:
        return img, labels

    H, W      = img.shape[:2]
    new_labels = list(labels)

    for _ in range(random.randint(1, n_paste)):
        crop_img, cls_id = random.choice(micro_crops)
        # Wider scale range for TACO small objects
        scale = random.uniform(0.02, 0.2)
        new_w = max(10, int(W * scale))
        new_h = max(10, int(H * scale))
        crop_r = cv2.resize(crop_img, (new_w, new_h))

        x1 = random.randint(0, max(0, W - new_w - 1))
        y1 = random.randint(0, max(0, H - new_h - 1))

        alpha = random.uniform(0.7, 1.0)
        roi    = img[y1:y1+new_h, x1:x1+new_w]
        img[y1:y1+new_h, x1:x1+new_w] = cv2.addWeighted(
            crop_r[:roi.shape[0], :roi.shape[1]], alpha,
            roi, 1 - alpha, 0
        )

        cx = (x1 + new_w / 2) / W
        cy = (y1 + new_h / 2) / H
        nw = new_w / W

```

```

        nh = new_h / H
        new_labels.append([cls_id, cx, cy, nw, nh])

    return img, new_labels

def extract_micro_crops(unified_dir, micro_class_ids=None):
    """
    Extracts bounding-box crops of micro-waste objects
    from existing training images for use in copy-paste.
    """
    if micro_class_ids is None:
        micro_class_ids = list(range(6, NUM_CLASSES)) # cls 6-9

    img_dir = Path(unified_dir) / 'train' / 'images'
    lbl_dir = Path(unified_dir) / 'train' / 'labels'
    crops = []

    for lbl_path in list(lbl_dir.glob('*.txt'))[:200]:
        img_path = img_dir / lbl_path.with_suffix('.jpg').name
        if not img_path.exists():
            continue
        img = cv2.imread(str(img_path))
        if img is None:
            continue
        H, W = img.shape[:2]

        with open(lbl_path) as f:
            for line in f:
                parts = line.strip().split()
                if not parts:
                    continue
                cid = int(parts[0])
                if cid not in micro_class_ids:
                    continue
                cx, cy, bw, bh = map(float, parts[1:])
                x1 = max(0, int((cx - bw/2) * W))
                y1 = max(0, int((cy - bh/2) * H))
                x2 = min(W, int((cx + bw/2) * W))
                y2 = min(H, int((cy + bh/2) * H))
                if x2 - x1 > 5 and y2 - y1 > 5:
                    crops.append((img[y1:y2, x1:x2].copy(), cid))

    print(f'Extracted {len(crops)} micro-waste crops for copy-paste')
    return crops

```

```

# =====
# 4.1 MSSA Module
# =====

class ChannelAttention(nn.Module):
    """Squeeze-and-Excitation channel attention (avg + max pool fusion)."""
    def __init__(self, in_channels, reduction=16):
        super().__init__()
        mid = max(1, in_channels // reduction)
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(in_channels, mid, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(mid, in_channels, bias=False),
        )
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = self.fc(self.avg_pool(x))
        max_out = self.fc(self.max_pool(x))
        return self.sigmoid(avg_out + max_out).unsqueeze(-1).unsqueeze(-1) * x

class SpatialAttention(nn.Module):
    """Spatial attention using avg + max pooled channel maps."""
    def __init__(self, kernel_size=7):
        super().__init__()
        pad = kernel_size // 2
        self.conv = nn.Conv2d(2, 1, kernel_size, padding=pad, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        attn = self.conv(torch.cat([avg_out, max_out], dim=1))
        return self.sigmoid(attn) * x

class MSSA(nn.Module):
    """
    Multi-Scale Spatial Attention Module.

    Processes features at 3 dilated scales:
    scale1 : 1x1 pointwise (local)
    """

```

```

    scale2 : 3x3 dilation=2      (medium receptive field)
    scale3 : 3x3 dilation=4      (large receptive field / global context)

```

*Then applies channel attention + spatial attention + residual connection.  
Injected into the YOLOv8 neck to enhance small-object detection.*

```

"""
def __init__(self, in_channels, reduction=16):
    super().__init__()
    mid = in_channels // 4

    self.scale1 = nn.Sequential(
        nn.Conv2d(in_channels, mid, 1, bias=False),
        nn.BatchNorm2d(mid), nn.SiLU()
    )
    self.scale2 = nn.Sequential(
        nn.Conv2d(in_channels, mid, 3, padding=2, dilation=2, bias=False),
        nn.BatchNorm2d(mid), nn.SiLU()
    )
    self.scale3 = nn.Sequential(
        nn.Conv2d(in_channels, mid, 3, padding=4, dilation=4, bias=False),
        nn.BatchNorm2d(mid), nn.SiLU()
    )

    self.fuse = nn.Sequential(
        nn.Conv2d(mid * 3, in_channels, 1, bias=False),
        nn.BatchNorm2d(in_channels),
    )

    self.channel_attn = ChannelAttention(in_channels, reduction)
    self.spatial_attn = SpatialAttention(kernel_size=7)
    self.act = nn.SiLU()

    def forward(self, x):
        s1 = self.scale1(x)
        s2 = self.scale2(x)
        s3 = self.scale3(x)
        fused = self.fuse(torch.cat([s1, s2, s3], dim=1))
        out = self.channel_attn(fused)
        out = self.spatial_attn(out)
        return self.act(out + x)  # residual

```

*# Sanity check*

```
from torchinfo import summary
```

```
mssa_test = MSSA(in_channels=256).to(DEVICE)
```

```

dummy      = torch.randn(2, 256, 40, 40).to(DEVICE)
out         = mssa_test(dummy)
print(f'MSSA output shape: {out.shape} (expected: torch.Size([2, 256, 40, 40]))')
print()
summary(mssa_test, input_size=(2, 256, 40, 40), device=DEVICE, verbose=0)

# =====
# 4.2 Visualise MSSA attention maps
# =====
def visualise_attention(mssa_module, img_tensor):
    """Shows what the spatial attention layer focuses on."""
    mssa_module.eval()
    activations = {}

    def hook_fn(module, input, output):
        activations['spatial'] = output.detach()

    hook = mssa_module.spatial_attn.conv.register_forward_hook(hook_fn)
    with torch.no_grad():
        _ = mssa_module(img_tensor)
    hook.remove()

    attn_map = activations['spatial'][0, 0].cpu().numpy()
    attn_norm = (attn_map - attn_map.min()) / (
        attn_map.max() - attn_map.min() + 1e-8)

    fig, axes = plt.subplots(1, 3, figsize=(13, 4))
    axes[0].imshow(img_tensor[0].mean(0).cpu().numpy(), cmap='gray')
    axes[0].set_title('Input Feature Map (mean)', fontweight='bold')
    axes[1].imshow(attn_norm, cmap='hot')
    axes[1].set_title('MSSA Spatial Attention', fontweight='bold')
    axes[2].imshow(img_tensor[0].mean(0).cpu().numpy(),
                    cmap='gray', alpha=0.5)
    axes[2].imshow(
        cv2.resize(attn_norm,
                    (img_tensor.shape[-1], img_tensor.shape[-2])),
        cmap='hot', alpha=0.6
    )
    axes[2].set_title('Overlay', fontweight='bold')
    for ax in axes:
        ax.axis('off')
    plt.suptitle('Multi-Scale Spatial Attention Visualisation',
                 fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()

```

```
dummy_vis = torch.randn(1, 256, 40, 40).to(DEVICE)
visualise_attention(mssa_test, dummy_vis)
```

```
# =====
# 5.1 Hybrid Focal-CIoU Loss
# =====

class FocalLoss(nn.Module):
    """
    Focal Loss: down-weights easy examples, focuses on hard / minority cases.
     $FL(p_t) = -\alpha_t * (1 - p_t)^\gamma * \log(p_t)$ 
    """
    def __init__(self, alpha=0.25, gamma=2.0, reduction='mean'):
        super().__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, pred, target):
        ce_loss = F.cross_entropy(pred, target, reduction='none')
        p_t = torch.exp(-ce_loss)
        focal_w = self.alpha * (1 - p_t) ** self.gamma
        loss = focal_w * ce_loss
        return loss.mean() if self.reduction == 'mean' else loss.sum()

def ciou_loss(pred_boxes, target_boxes, eps=1e-7):
    """
    Complete IoU Loss.
    Penalises: overlap area + centre distance + aspect ratio difference.
    Boxes in [x1, y1, x2, y2] format.
    """
    inter_x1 = torch.max(pred_boxes[:, 0], target_boxes[:, 0])
    inter_y1 = torch.max(pred_boxes[:, 1], target_boxes[:, 1])
    inter_x2 = torch.min(pred_boxes[:, 2], target_boxes[:, 2])
    inter_y2 = torch.min(pred_boxes[:, 3], target_boxes[:, 3])
    inter = (inter_x2 - inter_x1).clamp(0) * (inter_y2 - inter_y1).clamp(0)

    pred_area = ((pred_boxes[:, 2] - pred_boxes[:, 0]) *
                  (pred_boxes[:, 3] - pred_boxes[:, 1]))
    target_area = ((target_boxes[:, 2] - target_boxes[:, 0]) *
                   (target_boxes[:, 3] - target_boxes[:, 1]))
    union = pred_area + target_area - inter + eps
    iou = inter / union
```



```

enc_x1 = torch.min(pred_boxes[:,0], target_boxes[:,0])
enc_y1 = torch.min(pred_boxes[:,1], target_boxes[:,1])
enc_x2 = torch.max(pred_boxes[:,2], target_boxes[:,2])
enc_y2 = torch.max(pred_boxes[:,3], target_boxes[:,3])
c2      = (enc_x2-enc_x1)**2 + (enc_y2-enc_y1)**2 + eps

pred_cx  = (pred_boxes[:,0] + pred_boxes[:,2]) / 2
pred_cy  = (pred_boxes[:,1] + pred_boxes[:,3]) / 2
target_cx = (target_boxes[:,0] + target_boxes[:,2]) / 2
target_cy = (target_boxes[:,1] + target_boxes[:,3]) / 2
rho2      = (pred_cx-target_cx)**2 + (pred_cy-target_cy)**2

pred_w    = (pred_boxes[:,2]-pred_boxes[:,0]).clamp(eps)
pred_h    = (pred_boxes[:,3]-pred_boxes[:,1]).clamp(eps)
target_w  = (target_boxes[:,2]-target_boxes[:,0]).clamp(eps)
target_h  = (target_boxes[:,3]-target_boxes[:,1]).clamp(eps)
v         = (4/math.pi**2) * (
    torch.atan(target_w/target_h) -
    torch.atan(pred_w/pred_h))**2

with torch.no_grad():
    alpha_ciou = v / (1 - iou + v + eps)

ciou = iou - rho2/c2 - alpha_ciou * v
return (1 - ciou).mean()

class HybridFocalCIoULoss(nn.Module):
    """
    Hybrid Loss = beta * FocalLoss + (1 - beta) * CIoU Loss
    beta=0.6 means 60% weight on classification, 40% on box regression.
    """
    def __init__(self, alpha=0.25, gamma=2.0, beta=0.6):
        super().__init__()
        self.focal = FocalLoss(alpha=alpha, gamma=gamma)
        self.beta = beta

    def forward(self, cls_pred, cls_target, pred_boxes, target_boxes):
        fl = self.focal(cls_pred, cls_target)
        ciou = ciou_loss(pred_boxes, target_boxes)
        return self.beta * fl + (1 - self.beta) * ciou, fl.item(), ciou.item()

loss_fn = HybridFocalCIoULoss(alpha=0.25, gamma=2.0, beta=0.6)
bs = 8
cls_pred = torch.randn(bs, NUM_CLASSES).to(DEVICE)

```

```

cls_target = torch.randint(0, NUM_CLASSES, (bs,)).to(DEVICE)
pred_boxes = torch.rand(bs, 4).to(DEVICE)
target_boxes = torch.rand(bs, 4).to(DEVICE)
pred_boxes[:,2:] = pred_boxes[:,2:] + pred_boxes[:,2:].abs() + 0.01
target_boxes[:,2:] = target_boxes[:,2:] + target_boxes[:,2:].abs() + 0.01

total_loss, fl_val, ciou_val = loss_fn(
    cls_pred, cls_target, pred_boxes, target_boxes)
print(f'Hybrid Loss test:')
print(f'    Focal Loss   : {fl_val:.4f}')
print(f'    CIOU Loss    : {ciou_val:.4f}')
print(f'    Total Loss   : {total_loss.item():.4f}')

# =====
# 5.2 Focal loss visualisation
# =====

p = np.linspace(0.01, 0.99, 200)
gamma_vals = [0, 0.5, 1, 2, 5]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 5))

for g in gamma_vals:
    fl = -(1-p)**g * np.log(p)
    ax1.plot(p, fl, label=f'={g}', linewidth=2)
    ax1.set_xlabel('Probability of correct class (p_t)')
    ax1.set_ylabel('Loss')
    ax1.set_title('Focal Loss at different values', fontweight='bold')
    ax1.legend()
    ax1.grid(alpha=0.3)
    ax1.annotate('Easy examples\ndown-weighted here',
        xy=(0.8, 0.05), xytext=(0.55, 0.55),
        arrowprops=dict(arrowstyle='->', color='gray'),
        fontsize=9, color='gray')

counts_bar = [cls_counts.get(i, 0) for i in range(NUM_CLASSES)]
colors_bar = ['#e74c3c' if i >= 6 else '#3498db' for i in range(NUM_CLASSES)]
ax2.bar(WASTE_CLASSES, counts_bar, color=colors_bar, edgecolor='white')
ax2.set_title('Class Imbalance - Training Set\n(Focal Loss corrects this)',
    fontweight='bold')
ax2.set_ylabel('Instances')
ax2.tick_params(axis='x', rotation=45)
ax2.grid(axis='y', alpha=0.3)

plt.suptitle('Hybrid Focal-CIOU Loss - Motivation',
    fontsize=13, fontweight='bold')

```

```

plt.tight_layout()
plt.show()

# =====
# 6.1 Load pretrained YOLOv8m
# =====

print('Loading pretrained YOLOv8m (COCO weights)...')
print('  This downloads ~52 MB on the first run.')
print()

model = YOLO('yolov8m.pt')

n_params = sum(p.numel() for p in model.model.parameters())
print(f'YOLOv8m loaded')
print(f'  Parameters      : {n_params:,}')
print(f'  Default nc       : {model.model.nc}')
print(f'  Our nc            : {NUM_CLASSES}')

# =====
# 6.2 Inject MSSA into YOLOv8 neck
# =====

def inject_mssa(yolo_model, reduction=16):
    """
    Injects MSSA after each C2f block in the YOLOv8 neck (layers 11+).
    Returns the modified model.
    """
    from ultralytics.nn.modules import C2f

    seq      = yolo_model.model.model
    injected = []

    for idx, layer in enumerate(seq):
        if isinstance(layer, C2f) and idx >= 11:
            try:
                in_ch = layer.cv2.conv.out_channels
            except AttributeError:
                continue
            mssa      = MSSA(in_channels=in_ch, reduction=reduction).to(DEVICE)
            seq[idx] = nn.Sequential(layer, mssa)
            injected.append(idx)

    print(f'MSSA injected at neck layers: {injected}')
    n_params = sum(p.numel() for p in yolo_model.model.parameters())
    print(f'  Total parameters after injection: {n_params:,}')

```

```

        return yolo_model

model = inject_mssa(model)

# =====
# 6.3 Training configuration
# =====

TRAIN_CFG = dict(
    data = str(YAML_PATH),
    epochs = 100,          # Increased from 50
    imgsz = 1024,          # Increased from 640 for high-res images
    batch = -1,            # Auto-batch for H100 (optimizes VRAM usage)
    device = 0 if torch.cuda.is_available() else 'cpu',
    optimizer = 'AdamW',
    lr0 = 1e-3,
    lrf = 0.01,
    momentum = 0.937,
    weight_decay = 5e-4,
    warmup_epochs = 3,
    cos_lr = True,
    amp = True,            # Automatic Mixed Precision
    workers = 8,           # Increased workers for HPC
    close_mosaic = 10,     # Refine accuracy at the end
    # Augmentations - Optimized for bright/dense/small-object scenes
    hsv_h = 0.015,
    hsv_s = 0.6,           # Increased from 0.4
    hsv_v = 0.4,           # Increased from 0.2
    flipplr = 0.5,
    flipud = 0.0,
    mosaic = 1.0,
    mixup = 0.15,          # Increased from 0.1
    copy_paste = 0.4,      # Increased from 0.2
    degrees = 15.0,
    # Save and logging
    project = 'mssa_waste',
    name = 'mssa_yolov8m_optimized',
    save = True,
    save_period = 10,
    val = True,
    plots = True,
    verbose = True,
)

print('Training Configuration:')

```

```

for k, v in TRAIN_CFG.items():
    print(f'    {k:<20} = {v}')

# =====
LOCAL_CKPT_DIR = OUTPUTS_DIR / 'checkpoints'
LOCAL_LAST_PT  = LOCAL_CKPT_DIR / 'last.pt'
LOCAL_BEST_PT  = LOCAL_CKPT_DIR / 'best.pt'

LOCAL_CKPT_DIR.mkdir(parents=True, exist_ok=True)

def sync_checkpoint_local(trainer):
    """Sync latest weights to the checkpoints directory."""
    weights_dir = Path(trainer.save_dir) / 'weights'
    last        = weights_dir / 'last.pt'
    best        = weights_dir / 'best.pt'
    if last.exists():
        shutil.copy2(last, LOCAL_LAST_PT)
    if best.exists():
        shutil.copy2(best, LOCAL_BEST_PT)

def get_resume_checkpoint():
    """Returns path to last.pt if it exists for resuming."""
    return str(LOCAL_LAST_PT) if LOCAL_LAST_PT.exists() else None

print(f'Checkpoints will be saved to: {LOCAL_CKPT_DIR}')

# =====
# 6.4 Start training (with checkpoint resume + Drive sync)
# =====
resume_path = get_resume_checkpoint()

if resume_path:
    print(f'Resuming from: {resume_path}')
    model = YOLO(resume_path)
    model.add_callback('on_fit_epoch_end', sync_checkpoint_local)
    results = model.train(resume=True)
else:
    print('Starting fresh training run...')
    model.add_callback('on_fit_epoch_end', sync_checkpoint_local)
    results = model.train(**TRAIN_CFG)

# Clear VRAM after training
if torch.cuda.is_available():
    torch.cuda.empty_cache()

```

```

BEST_PT = list(Path('mssa_waste').rglob('best.pt'))
BEST_PT = str(BEST_PT[-1]) if BEST_PT else None

print()
print('Training complete!')
print(f'    Best weights : {BEST_PT}')
print(f'    Checkpoints  : {LOCAL_CKPT_DIR}')
if results and hasattr(results, 'save_dir'):
    print(f'    Results dir  : {results.save_dir}')

# =====
# 7.1 Load best checkpoint and evaluate
# =====

if BEST_PT and Path(BEST_PT).exists():
    print(f'Loading best model: {BEST_PT}')
    best_model = YOLO(BEST_PT)
else:
    print('best.pt not found - using current model for evaluation')
    best_model = model

print()
print('Running evaluation on validation set...')
val_results = best_model.val(
    data      = str(YAML_PATH),
    split     = 'val',
    imgsz     = 640,
    batch     = 16,
    device    = 0 if torch.cuda.is_available() else 'cpu',
    plots     = True,
    save_json = True,
    verbose   = True,
)

print()
print('Results:')
print(f'    mAP50      : {val_results.box.map50:.4f}')
print(f'    mAP50-95   : {val_results.box.map:.4f}')
print(f'    Precision  : {val_results.box.mp:.4f}')
print(f'    Recall     : {val_results.box.mr:.4f}')

# =====
# 7.2 Per-class mAP breakdown
# =====

```

```

try:
    maps = val_results.box.maps
    if maps is not None and len(maps) == NUM_CLASSES:
        fig, ax = plt.subplots(figsize=(12, 5))
        colors = ['#e74c3c' if i >= 6 else '#3498db' for i in range(NUM_CLASSES)]
        bars = ax.bar(WASTE_CLASSES, maps,
                      color=colors, edgecolor='white', linewidth=0.8)
        ax.set_title('Per-Class mAP@50', fontsize=14, fontweight='bold')
        ax.set_ylabel('mAP@50')
        ax.set_ylim(0, 1.05)
        ax.tick_params(axis='x', rotation=45)
        ax.axhline(y=val_results.box.map50, color='black',
                  linestyle='--', linewidth=1.5,
                  label=f'Mean mAP50 = {val_results.box.map50:.3f}')
        ax.legend(handles=[
            mpatches.Patch(color='#3498db', label='Macro waste'),
            mpatches.Patch(color='#e74c3c', label='Micro waste'),
            plt.Line2D([0],[0], color='black', ls='--',
                      label=f'Mean mAP50 = {val_results.box.map50:.3f}')
        ])
        for bar, v in zip(bars, maps):
            ax.text(bar.get_x() + bar.get_width()/2, v + 0.01,
                  f'{v:.2f}', ha='center', va='bottom', fontsize=9)
        plt.tight_layout()
        plt.savefig(str(OUTPUTS_DIR / 'per_class_map.png'), dpi=150, bbox_inches='tight')
        plt.show()
    else:
        print('Per-class mAP not available - train on more data.')
except Exception as e:
    print(f'Per-class breakdown unavailable: {e}')

# =====
# 7.3 Training curves
# =====

results_csv = list(Path('mssa_waste').rglob('*.csv'))

if results_csv:
    df = pd.read_csv(results_csv[-1])
    df.columns = [c.strip() for c in df.columns]

    fig, axes = plt.subplots(2, 3, figsize=(16, 9))
    axes = axes.flatten()

    metrics = [

```

```

        ('train/box_loss',      'Box Loss (train)',      '#e74c3c'),
        ('train/cls_loss',      'Cls Loss (train)',      '#e67e22'),
        ('metrics/mAP50(B)',     'mAP@50',          '#2ecc71'),
        ('metrics/mAP50-95(B)',  'mAP50-95',        '#3498db'),
        ('metrics/precision(B)', 'Precision',        '#9b59b6'),
        ('metrics/recall(B)',    'Recall',           '#1abc9c'),
    ]

    for ax, (col, title, color) in zip(axes, metrics):
        if col in df.columns:
            ax.plot(df.index + 1, df[col], color=color, linewidth=2)
            ax.set_title(title, fontweight='bold')
            ax.set_xlabel('Epoch')
            ax.grid(alpha=0.3)
        else:
            ax.set_title(f'{title} (not available)', color='gray')
            ax.axis('off')

    plt.suptitle('MSSA-YOLOv8 Training Curves', fontsize=15, fontweight='bold')
    plt.tight_layout()
    plt.savefig(str(OUTPUTS_DIR / 'training_curves.png'), dpi=150, bbox_inches='tight')
    plt.show()
    print(f'Curves saved → {OUTPUTS_DIR}/training_curves.png')
else:
    print('results.csv not found. Run training first (Step 6).')

# =====
# 8.1 Inference helpers
# =====
COCO_MODEL = YOLO('yolov8m.pt')  # pretrained on COCO 80 classes

NON_WASTE_COCO_IDS = {
    14, 15, 16, 17, 18, 19, 20, 21, 22, 23,  # animals
    0,                                         # person
    1, 2, 3, 4, 5, 6, 7, 8,                  # vehicles
    56, 57, 58, 59, 60, 61, 62, 63, 64, 65,  # furniture
    46, 47, 48, 49, 50, 51, 52, 53, 54, 55,  # food
}

CLASS_COLORS = {name: tuple(CLASS_COLORS_BGR[i]) for i, name in enumerate(WASTE_CLASSES)}

def iou(boxA, boxB):
    """Compute IoU between two [x1,y1,x2,y2] boxes."""
    xA = max(boxA[0], boxB[0]); yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2]); yB = min(boxA[3], boxB[3])

```



```

inter = max(0, xB - xA) * max(0, yB - yA)
if inter == 0:
    return 0.0
aA = (boxA[2]-boxA[0]) * (boxA[3]-boxA[1])
aB = (boxB[2]-boxB[0]) * (boxB[3]-boxB[1])
return inter / (aA + aB - inter)

def run_inference(yolo_model, img_path, conf_thresh=0.25, iou_thresh=0.45,
                  coco_conf=0.4, coco_iou_gate=0.3):
    """
    Run waste detection with COCO gating.
    Any waste detection overlapping (IoU > coco_iou_gate) a high-confidence
    COCO non-waste object is suppressed.
    """
    device = 0 if torch.cuda.is_available() else 'cpu'

    results = yolo_model.predict(
        source=str(img_path), conf=conf_thresh,
        iou=iou_thresh, imgsz=640, device=device, verbose=False
    )[0]

    coco_results = COCO_MODEL.predict(
        source=str(img_path), conf=coco_conf,
        iou=iou_thresh, imgsz=640, device=device, verbose=False
    )[0]

    coco_non_waste_boxes = []
    for box in coco_results.bboxes:
        cid = int(box.cls)
        if cid in NON_WASTE_COCO_IDS:
            coco_non_waste_boxes.append(box.xyxy[0].tolist())

    img = cv2.imread(str(img_path))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    detections = []

    for box in results.bboxes:
        cls_id = int(box.cls)
        cls_name = WASTE_CLASSES[cls_id] if cls_id < NUM_CLASSES else str(cls_id)
        conf = float(box.conf)
        x1, y1, x2, y2 = map(int, box.xyxy[0].tolist())
        waste_box = [x1, y1, x2, y2]

        suppressed = any(
            iou(waste_box, coco_box) > coco_iou_gate
            for coco_box in coco_non_waste_boxes

```

```

    )
    if suppressed:
        continue

    color = CLASS_COLORS.get(cls_name, (200, 200, 200))
    cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
    label = f'{cls_name} {conf:.2f}'
    (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 2)
    cv2.rectangle(img, (x1, y1 - th - 6), (x1 + tw + 4, y1), color, -1)
    cv2.putText(img, label, (x1 + 2, y1 - 4),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 0), 2)
    detections.append({'class': cls_name, 'conf': conf, 'box': waste_box})

return img, detections

# =====
# 8.2 Inference example
# =====
# Note: For HPC servers, you can run this block by specifying a path to an image.
# Example:
# test_path = PROJECT_ROOT / 'test.jpg'
# if test_path.exists():
#     annotated, dets = run_inference(best_model, test_path, conf_thresh=0.2)
#     ...
print("Inference helper ready. Use run_inference(model, img_path) to test on new images.")

```